SYSTEM DESIGN · INTERVIEW NOTES

Interview mein poocha:

Explain *consistent hashing.*

Why `hash % N` breaks the day you add one server, how the ring fixes it, the two problems the ring still has, and the 60-second answer that gets you Selected.



1 The problem with hash % N

2 The ring

3 Two problems → virtual nodes

4 Code: 25 lines

5 The 60-second answer + follow-ups

6 Cheat sheet

hash(key) % N *works*, until N changes.

You have 10 cache servers. To find where a key lives: `server = hash(key) % 10`. Fast, simple, evenly spread.

```
hash("user:42") = 1234
1234 % 10 = 4  → cache 4
1234 % 11 = 2  → cache 2 (miss!)
```

Add an 11th server and the formula changes for **almost every key**. A key keeps its server only if `h % 10 == h % 11`, which happens for roughly 1 key in 11.

~91% of keys move when you go from 10 → 11 servers.
In general ≈ N/(N+1) of keys move.

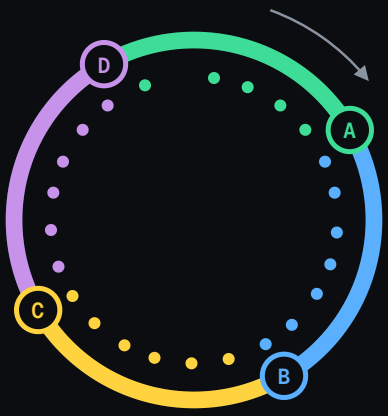
Why that kills the database

Every moved key is now a cache **miss**. Misses fall through to the database at the same moment, so DB reads jump many times over (at a 99% hit rate, misses go from ~1% to ~90% of requests) and it can go down. You added capacity and caused an outage.

> **Interview tip:** say the number. "Adding one server remaps about 90% of keys" is what shows you understand the problem, not just the definition.

Put servers *and* keys on the same circle.

1. Hash space is a circle: `0 … 2³²−1`, wrapping around.
2. Hash each **server** (e.g. its IP) to a point on the circle.
3. Hash each **key** to a point too.
4. Owner = the first server you meet walking **clockwise** from the key.



4 servers. Each owns the arc behind it.

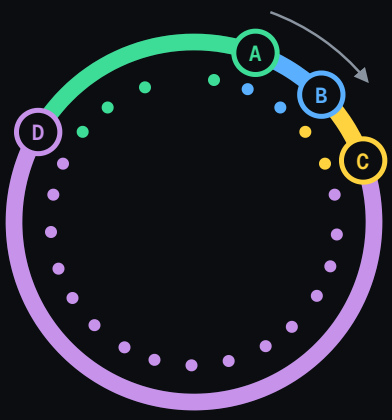


Add E: only keys on E's arc move (bright dots).

~1/(N+1) of keys move when a server is added: only the slice the new server takes over. Everyone else stays put.

Remove a server? Its keys just go to the next server clockwise. Again, nobody else is touched.

The basic ring is *not* fair.

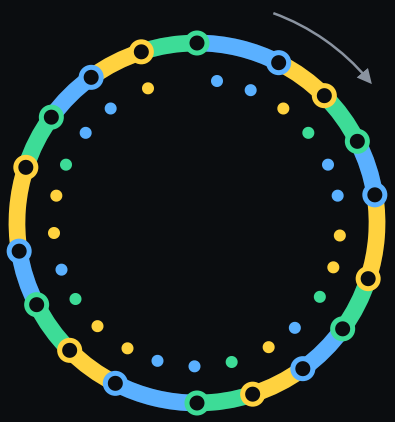


Problem 1 · Uneven arcs. Random positions can bunch up. Here D owns ~230° of the circle; B and C own just 25° each.

Problem 2 · Failure dumps on one neighbour.

When a server dies, **all** its keys go to the single next server. That neighbour's load can double, and it can fall over next: a cascade.

Fix: *virtual nodes.*



3 servers × 6 points each. Colours interleave around the ring.

Hash each server to **many** points: `hash("A#1")`, `hash("A#2")` …

- **Even load:** many small arcs average out.
- **Failure spreads:** a dead server's arcs go to *many* different neighbours.
- **Weights:** a 2× bigger machine gets 2× the points.

Cassandra's `num_tokens` default was 256; 4.0 lowered it to 16 with a smarter token allocator.

The whole idea in *25 lines.*

```
import bisect, hashlib

def h(s):  # stable 32-bit hash
    return int(hashlib.md5(s.encode()).hexdigest()[:8], 16)

class Ring:
    def __init__(self, vnodes=100):
        self.v = vnodes
        self.points = []   # sorted hash positions
        self.owner = {}    # position -> server

    def add(self, server):
        for i in range(self.v):
            p = h(f"{server}#{i}")
            bisect.insort(self.points, p)
            self.owner[p] = server

    def remove(self, server):
        for i in range(self.v):
            p = h(f"{server}#{i}")
            self.points.remove(p)
            del self.owner[p]

    def get(self, key):
        i = bisect.bisect(self.points, h(key))
        return self.owner[self.points[i % len(self.points)]]
```

- **Lookup** = binary search on a sorted list: `O(log(N·V))`.
- `i % len` handles the wrap-around past the top of the circle.
- Use a **stable** hash (md5, murmur, xxhash), never Python's built-in `hash()`, which changes between runs.
- Real systems also handle two vnodes colliding on the same point; skipped here.

The *60-second* answer.

"With `hash % N`*, adding or removing one server remaps about N/(N+1) of keys, so a cache cluster gets a miss storm that can take down the database.*

Consistent hashing puts servers and keys on the same hash ring. A key belongs to the first server clockwise. Adding a server only moves the keys in its new arc, about 1/N of them.

The plain ring gives uneven arcs, and a failed node dumps its whole load on one neighbour. Virtual nodes fix both: each server gets many points, so load evens out and failures spread. You can weight bigger machines with more points.

Lookup is a binary search over sorted positions. Dynamo and Cassandra use this."

Follow-ups they ask

Q: Replication? Walk clockwise and store on the next R *distinct physical* servers (skip vnodes of the same machine). Dynamo calls this the preference list.

Q: One hot key (a celebrity)? The ring doesn't fix that. One key always maps to one place. Replicate it, cache it locally, or split it into sub-keys.

Q: Alternatives? Rendezvous (HRW) hashing: pick the server with the highest `hash(key, server)`. Jump hash (Google, 2014): tiny and fast, but servers must be numbered 0…N−1. Bounded-load consistent hashing caps how full any server gets.

Save *this* page.

	HASH % N	RING	RING + VNODES
Keys moved on +1 server	~N/(N+1)	~1/(N+1)	~1/(N+1)
Load balance	even	can be skewed	even
Server dies	everything moves	1 neighbour takes all	spread out
Bigger machines	no	no	more vnodes
Lookup	O(1)	O(log N)	O(log N·V)

Where you'll see it

Amazon Dynamo · Apache Cassandra token ring · client-side cache sharding (e.g. ketama for memcached) · load balancers with ring-hash / Maglev policies (e.g. Envoy).